

Test/Simulate your System Verilog

Given: I already made my System Verilog Module + Testbench code.

Purpose: Test my code  and show how it works .

(1. How to test + simulate my System Verilog code): ✨

Note: do this in regular CMD on windows

1. `cd C:\Users\vanes\OneDrive\Documents\VLSI_FIR_ACCELERATOR` // locate it
2. `iverilog -g2012 -o fir_test fir_accelerator.sv fir_testbench.sv` // compile my module + tb files, (fir_test = arbitrary name)
3. `vvp fir_test` // test my code
+ create simulation
4. `gtkwave fir_waveform.vcd` // open GTK Wave

Result: 😊

```
PS C:\Users\vanes\OneDrive\Documents\VLSI_FIR_ACCELERATOR> iverilog -g2012 -o fir_test fir_accelerator.sv fir_testbench.sv
PS C:\Users\vanes\OneDrive\Documents\VLSI_FIR_ACCELERATOR> vvp fir_test
VCD info: dumpfile fir_waveform.vcd opened for output.
Test 1: y =      12
TEST 1 PASSED
Test 2: y =       8
TEST 2 PASSED
Test 3: y =       9
TEST 3 PASSED
Test 4: y =       7
TEST 4 PASSED
fir_testbench.sv:88: $finish called at 40 (1s)
PS C:\Users\vanes\OneDrive\Documents\VLSI_FIR_ACCELERATOR> gtwave fir_waveform.vcd

GTKWave Analyzer v3.3.100 (w)1999-2019 BSI

[0] start time.
[40] end time.
```

PS notice the values next to the output, y, it's literally hexadecimal for the actual answers (12, 8, 9, 7). Also, I will be showing you all my commands IN ORDER to make this stuff possible.

SST

fir_testbench
└─ dut

Type	Signals
wire	h0[7:0]
wire	h1[7:0]
wire	h2[7:0]
wire	x0[7:0]
wire	x1[7:0]
wire	x2[7:0]
reg	y[19:0]

Filter:

Append

Insert

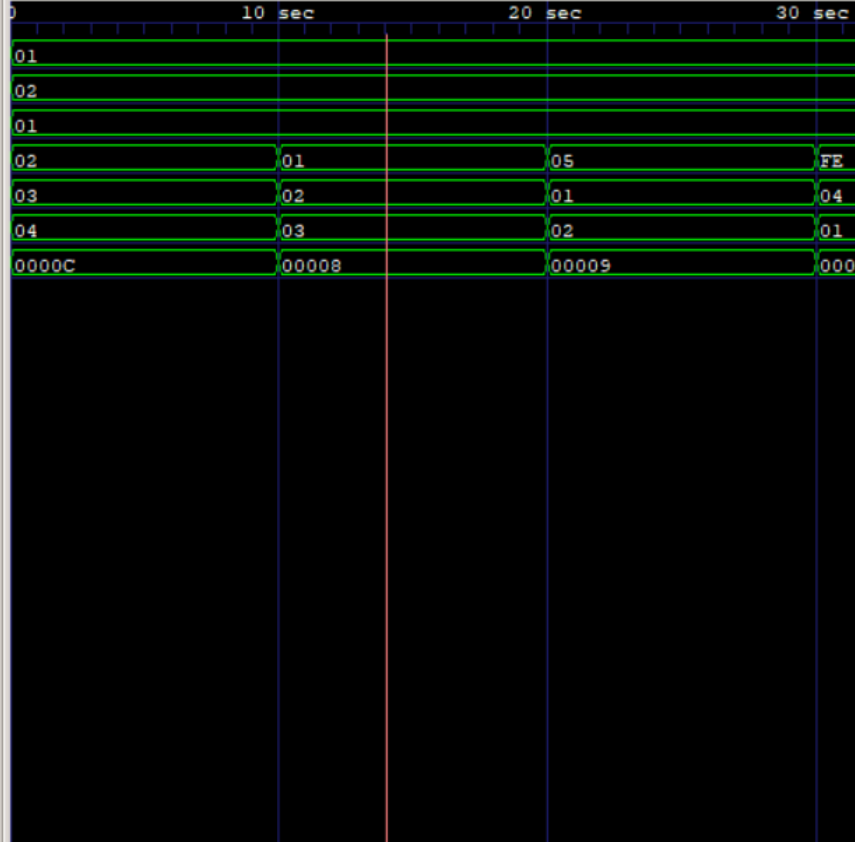
Replace

Signals

Time

h0[7:0] = 01
h1[7:0] = 02
h2[7:0] = 01
x0[7:0] = 01
x1[7:0] = 02
x2[7:0] = 03
y[19:0] = 00008

Waves



Logical Chip --> Physical Chip
Parameters

Given: I already made my System Verilog Module + Testbench code.

Purpose: I'm going to make my design for an imaginary computer chip into a worthy physical fabrication design for a chip.

Note: do this on Ubuntu

How to synthesize my SystemVerilog code with Yosys: (synthesize = to take my code and convert it to logical/non-physical chip parameters)

First, enter the OpenLane container: (2. create my "stuff" list, whatever I want to add to the linux room)

1. cd ~/OpenLane // locate OpenLane
2. make mount // enter the OpenLane Docker environment where Yosys + OpenROAD are installed
3. ls /home/vanes/fir_synthesis // verify that fir_accelerator.sv is visible (fir_synthesis = arbitrary name for folder to keep RTL)

Then start Yosys:

4. yosys // open the Yosys synthesis environment

Inside the yosys> prompt: (3. convert my code → imaginary hardware (logical cells))

5. read_verilog -sv /home/vanes/fir_synthesis/fir_accelerator.sv // load our SystemVerilog RTL
6. hierarchy -top fir_accelerator // tell Yosys which module is the top-level design
7. proc // convert procedural RTL such as always_comb into hardware representation
8. opt // optimize and remove unnecessary hardware
9. stat // display statistics about our synthesized design

```
5. Printing statistics.

=== fir_accelerator ===

Number of wires:          11
Number of wire bits:      148
Number of public wires:   7
Number of public wire bits: 68
Number of memories:       0
Number of memory bits:    0
Number of processes:      0
Number of cells:          5
  $add                    2
  $mul                     3
```

(4. Now that you created the container, add my code + some new rules to it):

Note: go back to container (CTRL + C)

1. mkdir -p designs/fir_accelerator/src // create the OpenLane design folder and source-code folder

```

2. cp /home/vanes/fir_synthesis/fir_accelerator.sv designs/fir_accelerator/src/ // copy our
   SystemVerilog RTL into the OpenLane
3. ls designs/fir_accelerator/src // verify that fir_accelerator.sv is there
4. cat > designs/fir_accelerator/config.json <<'EOF' // create the OpenLane configuration file
   {
       "DESIGN_NAME": "fir_accelerator",
       "VERILOG_FILES": "dir::src/*.sv",
       "pdk::sky130*": {
           "FP_CORE_UTIL": 30,
           "CLOCK_PERIOD": 12
       }
   }
   EOF
5. cat designs/fir_accelerator/config.json // display the configuration and verify that it
   was created correctly

```

(5. Run all my backend stuff so you can make real chip)

Note: This will likely take you 2-3 minutes on windows. Wait until 42.

```

6. ./flow.tcl -design fir_accelerator // run the OpenLane RTL-to-GDS flow

```

(6. Actually present all the files/results, EXCEPT for final chip view - next!)

```

8. ls designs/fir_accelerator/runs // show the OpenLane
   runs that were created
9. ls designs/fir_accelerator/runs/RUN_2026.08.17_07.50.54/results/final // show the
   final GDS, DEF, LEF, and netlist files
10. cat designs/fir_accelerator/runs/RUN_2026.08.17_07.50.54/reports/metrics.csv //
    inspect the final design metrics

```

```

ns/RUN_2026.08.17_03.56.10/reports/manufacturability.rpt'.
[INFO]: Created metrics report at 'designs/fir_accelerator/run_2026.08.17_03.56.10/reports/metrics.csv'.
[INFO]: There are no max slew, max fanout or max capacitance
in the design at the typical corner.
[INFO]: There are no hold violations in the design at the typical
corner.
[INFO]: There are no setup violations in the design at the typical
corner.
[SUCCESS]: Flow complete.
[INFO]: Note that the following warnings have been generated:
[WARNING]: 1 warnings found by linter
[WARNING]: Module sky130_fd_sc_hd__tapvpwrvrgnd_1 blackboxed during
[WARNING]: Module sky130_fd_sc_hd__fill_2 blackboxed during synthesis
[WARNING]: Module sky130_fd_sc_hd__fill_1 blackboxed during synthesis
[WARNING]: Module sky130_ef_sc_hd__decap_12 blackboxed during synthesis
[WARNING]: VSRC_LOC_FILES is not defined. The IR drop analysis was
performed, but the values may be inaccurate.

OpenLane Container (ff5509f):/openlane$ cat designs/fir_accelerator/fig.json
{
  "DESIGN_NAME": "fir_accelerator",
  "VERILOG_FILES": "dir::src/*.sv",
  "CLOCK_PORT": "__VIRTUAL_CLK__",
  "CLOCK_PERIOD": 12,
  "pdk::sky130*": {
    "FP_CORE_UTIL": 30
  }
}

```

Definitions regarding the files of the completed physical design:

GDS = gds

The big one for us. This is the physical layout of my chip, the geometry of cells, metal, vias, layers, etc. This is the file we opened with KLayout. (fir_accelerator.gds)

DEF = def

A description of the physical design, including placement and routing information. Think of it as a detailed textual representation of where things are physically placed and connected.

LEF = lef

A physical abstract used by the place-and-route tools. It describes things like cell dimensions, pins, and routing information without containing all the detailed geometry of the GDS.

Timing / electrical information

LIB = lib

Library timing/power information for the standard cells.

SDC = sdc

My timing constraints, things like the clock definition and timing requirements.

SDF = sdf

Timing delay information that can be used for timing-aware simulation.

SPEF = spef

Parasitic information extracted from the physical layout, mainly the resistance and capacitance associated with the interconnect.

Other physical/circuit representations

MAG = mag

A Magic layout representation of the design.

MAGLEF = maglef

A Magic-related LEF/abstract representation.

SPI = spi

A SPICE-format circuit representation, useful for circuit-level analysis.

Verilog = verilog

The final Verilog netlist representing the implemented logic.

```
OpenLane Container (ff5509f):/openlane$ ls -lt designs/fir_accelerations
total 12
drwxr-xr-x 6 1000 1000 4096 Aug 17 03:58 RUN_2026.08.17_03.56.10
drwxr-xr-x 6 1000 1000 4096 Aug 17 03:51 RUN_2026.08.17_03.49.20
drwxr-xr-x 7 1000 1000 4096 Aug 17 03:46 RUN_2026.08.17_03.46.30
OpenLane Container (ff5509f):/openlane$ ls designs/fir_accelerations/RUN_2026.08.17_03.56.10/results/final
def gds lef lib mag maglef sdc sdf spef spi verilog
```


Metric	Result	
Flow status	flow completed	// OpenLane process completed successfully? Yes
Total runtime	2 min 22 sec	// How long that took
Routing runtime	1 min 52 sec	// How long it took the computer to even figure out where to wire stuff
Die area	0.052245 mm ²	// die and core area are “how much silicon am I using”
Core area	44,990.65 μm ²	// area inside the die area (logic area)
Synthesized cells	1,339	
Total cells	5,653	// physical chip has LOTS more cells than a logical (online) chip
Inputs	54	
Outputs	178	
Wire length	41,972	// connect the cells together
Vias	9,712	// how signals connect between layers of cells inside metal
Critical path	6.54 ns	// slowest (max budget) signal travel through design
Suggested clock period	12 ns	// maximum clock period budget
Suggested frequency	83.33 MHz	// 1/12ns (think)
Target clock period	12 ns	// I did better
Core utilization	30%	// how much of my chip is covered by cells/logic
Standard-cell library	sky130_fd_sc_hd	// framework for our physical cell

Or my not-so-pretty-terminal shows this: 😞

```
OpenLane Container (ff5509f):/openlane$ cat designs/fir_acceler
s/RUN_2026.08.17_03.56.10/reports/metrics.csv
design,design_name,config,flow_status,total_runtime,routed_r
l/mm^2)/Core_Util,DIEAREA_mm^2,CellPer_mm^2,OpenDP_Util,Fina
_Memory_Usage_MB,synth_cell_count,tritonRoute_violations,Sho
ns,MetSpc_violations,OffGrid_violations,MinHole_violations,O
ions,Magic_violations,pin_antenna_violations,net_antenna_vio
_total_errors,cvc_total_errors,klayout_violations,wire_lengt
pl_wns,optimized_wns,fastroute_wns,spef_wns,tns,pl_tns,optim
stroute_tns,spef_tns,HPWL,routing_layer1_pct,routing_layer2_
_layer3_pct,routing_layer4_pct,routing_layer5_pct,routing_la
res_count,wire_bits,public_wires_count,public_wire_bits,memor
memory_bits,processes_count,cells_pre_abc,AND,DFF,NAND,NOR,O
MUX,inputs,outputs,level,DecapCells,WelltapCells,DiodeCells,
onPhysCells,TotalCells,CoreArea_um^2,power_slowest_internal_
lowest_switching_uW,power_slowest_leakage_uW,power_typical_in
ower_typical_switching_uW,power_typical_leakage_uW,power_fas
al_uW,power_fastest_switching_uW,power_fastest_leakage_uW,cr
_ns,suggested_clock_period,suggested_clock_frequency,CLOCK_P
PECT_RATIO,FP_CORE_UTIL,FP_PDN_HPITCH,FP_PDN_VPITCH,GRT_ADJU
REPAIR_ANTENNAS,MAX_FANOUT_CONSTRAINT,PL_TARGET_DENSITY,RUN_
IODE_INSERTION,STD_CELL_LIBRARY,SYNTH_STRATEGY
/openlane/designs/fir_accelerator,fir_accelerator,RUN_2026.0
10,flow completed,0h2m22s0ms,0h1m52s0ms,91491.59967987415,0.
5,27447.479903962245,30.99,-1,572.77,1339,0,0,0,0,0,0,0,3,3,
72,9712,-0.04,-1,-1,-1,0.0,-0.07,-1,-1,-1,0.0,35594756.0,0.0
8,1.08,0.22,0.0,1543,1604,7,68,0,0,0,1554,153,0,70,54,178,41
18,22,2715,640,1,863,1434,5653,44990.6496,-1,-1,-1,0.00116,0
e-09,-1,-1,-1,6.539999999999999,12.0,83.33333333333333,12,1,
53.6,0.3,1,10,0.4,0,sky130_fd_sc_hd,AREA 0
```

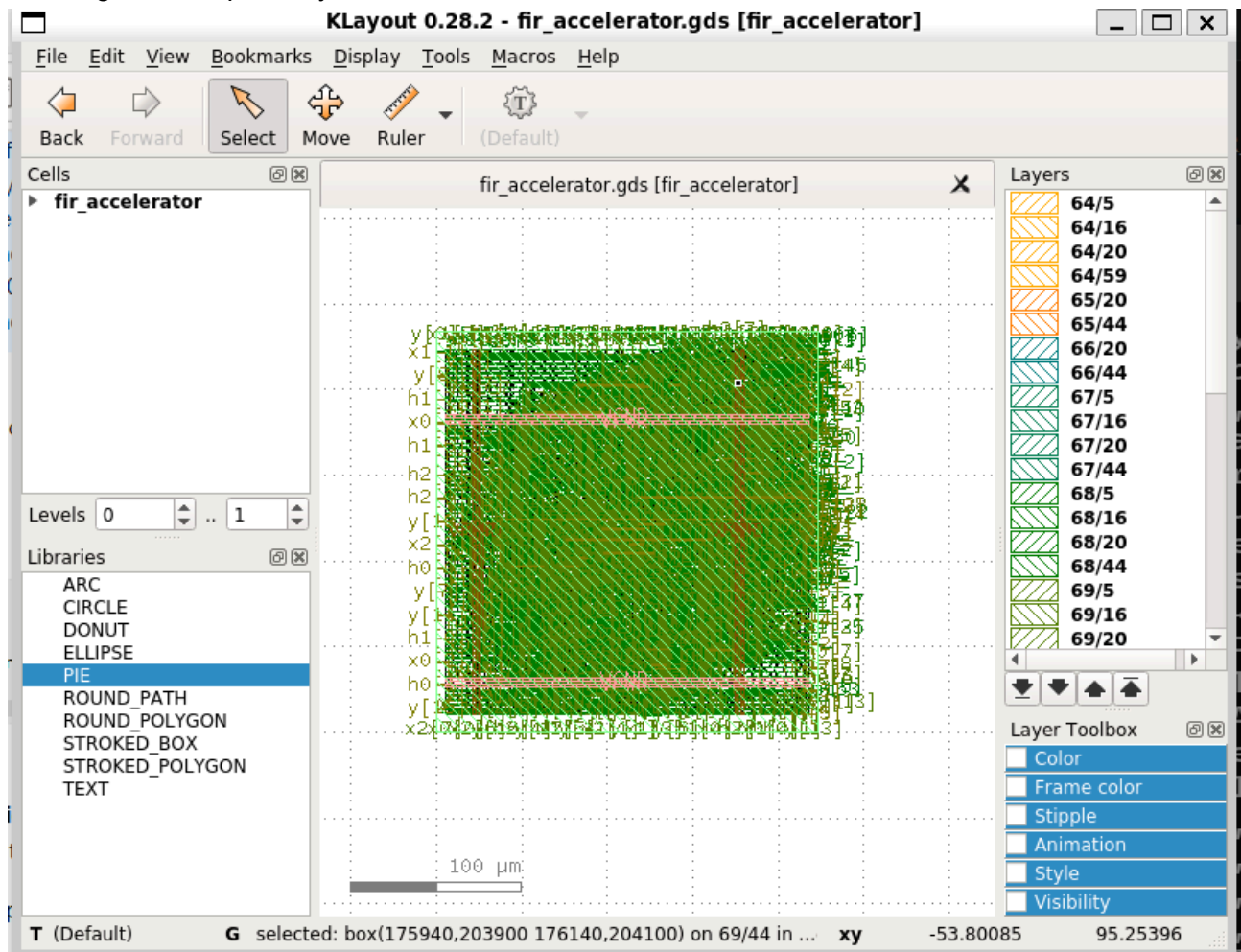
Displaying our PHYSICAL Chip

Given: all the hard work

Purpose: Show what it gets ya

(7. View the final physical layout):

1. Is designs/fir_accelerator/runs/RUN_2026.08.17_03.56.10/results/final/gds // display whatever file you want (gds, def, etc)
2. layout -v // prepare layout
3. layout designs/fir_accelerator/runs/RUN_2026.08.17_03.56.10/results/final/gds/fir_accelerator.gds // open layout



Zoomed in view:

- Notice the Sky130 marks. Our backend used the Sky130 standards to make our circuit “physically ready” for fabrication.



KLayout 0.28.2 - fir_accelerator.gds [fir_accelerator]



File Edit View Bookmarks Display Tools Macros Help



Back



Forward



Select



Move



Ruler



(Default)

Cells

► fir_accelerator

Levels

0



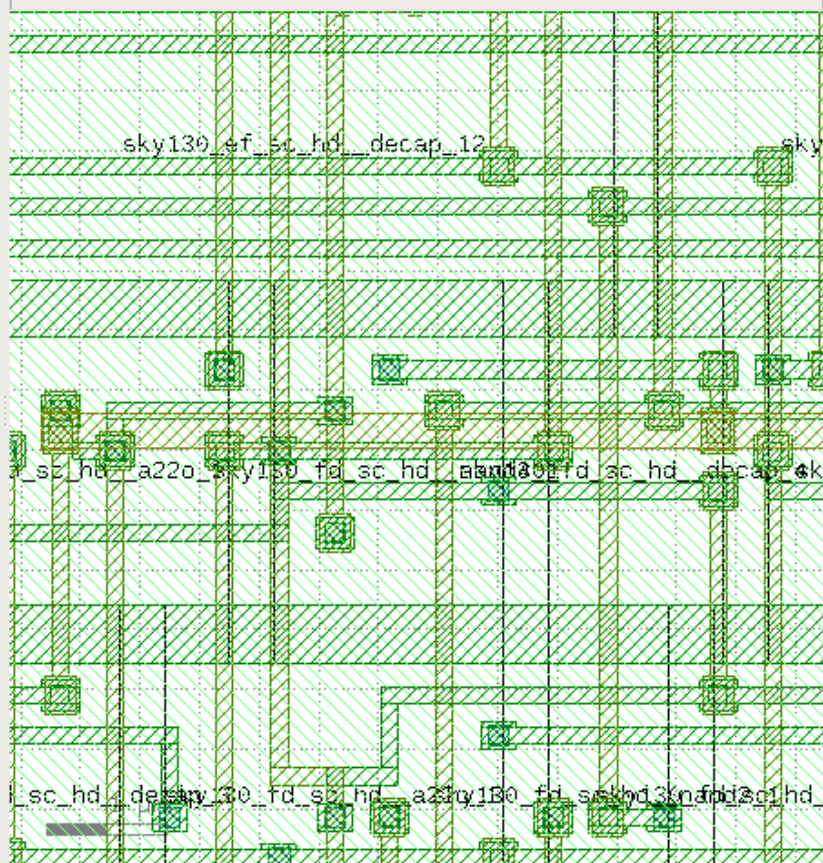
..

1

Libraries

ARC
CIRCLE
DONUT
ELLIPSE
PIE
ROUND_PATH
ROUND_POLYGON
STROKED_BOX
STROKED_POLYGON
TEXT

fir_accelerator.gds [fir_accelerator]



Layers

64/5
64/16
64/20
64/59
65/20
65/44
66/20
66/44
67/5
67/16
67/20
67/44
68/5
68/16
68/20
68/44
69/5
69/16
69/20



Layer Toolbox

Color
Frame color
Stipple
Animation
Style
Visibility

T (Default)

G selected: box(175940,203900 176140,204100) on 69/44 in ... xy

103.21587

116.08926

Connecting the dots...

By now you're likely really glad that you found someone who can easily explain this. 🤗 But just in case you still don't understand this, here's a tree. 🌳 (one with more leaves)

Vanessa Knight wrote this.

FIR - Small specialized Hardware Accelerator

FIR = type of filter

RTL = hardware explanation of what system Verilog code does

↳ how does it do it? $y = x_0 h_0 + x_1 h_1 + x_2 h_2 + \dots$

① Write system Verilog code to describe "y"

② Test + simulate via GTKWave

↳ Test 1: $y=12$; TEST 1 PASSED

↳ ...

y ————— 0000c ← some hexadecimal value

③ Set up OpenLane container with verilog system MODULE

④ Start Yosys and load RTL, convert to hardware info

↳ # cells = 5, # add = 2, # mul = 3, # wires

⑤ Go back to OpenLane container and add Rules for our chip

↳ clock period, design name

↳ create json package

⑥ Run ./flow.tcl -design fir-accelerator

↳ runs all backend stuff (synthesis, floorplan, placement...)

↳ returns measurements (Die area, total cells, core area...)

Note Yosys output: said 5 cells and OpenLane container output returns ≈ 5000 cells. 5 cells are "logical" cells, 2 add gates, and 3 multiplier gates. 5000 cells are those 5 cells mapped onto SKY30 (fabrication standard), "physical" cells.

⑦ create GDS + KLayout file; View

① System Verilog code
② Theoretical Simulation
③ → ④ → ⑤ + backend → ⑦ Final chip

UR TREE
to success



to bed, it's midnight again.